Informe Consolidado de Pruebas de Software

E-commerce VibePulse - Consolidacion QA de informes de pruebas

Equipo de Ingenieria de Software

27 de abril de 2026

Control del documento

Campo	Detalle
Proyecto	E-commerce VibePulse
Tipo de documento	Informe consolidado de pruebas de software
Version	1.0
Fecha	27 de abril de 2026
Elaborado para	Equipo de 5 ingenieros de software
Alcance	Consolidacion de hallazgos derivados de 10 informes de pruebas
Exclusiones	No se incluyen defectos responsive/mobile porque fueron reportados como solucionados
Criterio usado	Consolidacion sin redundancia: un defecto repetido en varios informes se registra una sola vez

Resumen ejecutivo

Este documento consolida los hallazgos de calidad identificados en los informes de pruebas del sistema **E-commerce VibePulse**. La revision integra resultados de pruebas de humo, pruebas API, pruebas de integracion, pruebas E2E, pruebas de rendimiento y carga, analisis estatico de codigo, revision estatica de requisitos y diseno, accesibilidad y cobertura de codigo.

El sistema presenta una base funcional solida en backend, autenticacion, rutas protegidas, integracion API y smoke testing. Las pruebas de humo y las pruebas API del modulo de autenticacion fueron aprobadas. Las pruebas de integracion del backend y las pruebas E2E basicas tambien muestran estabilidad. Sin embargo, la revision consolidada evidencia riesgos importantes en configuracion de pruebas, rate limiting, cobertura insuficiente,

accesibilidad, validaciones, seguridad estatica, gestion de inventario, definicion incompleta de requisitos y trazabilidad.

La estructura de este informe se inspira en practicas de reporte de calidad usadas en herramientas de gestion de pruebas como IBM Engineering Test Management, donde se reportan cobertura, estado de prueba, veredictos, resultados pasados, fallidos, diferidos y registros de error. Tambien se alinea con la nocion de informe resumen de pruebas de ISTQB, entendido como un documento que resume actividades, resultados y evaluacion de los elementos probados contra criterios de salida.

Veredicto global

El sistema es funcionalmente estable en sus flujos base, pero no debe considerarse listo para una entrega final de calidad sin cerrar los hallazgos criticos y altos descritos en este informe.

Area evaluada	Resultado consolidado	Veredicto QA
Smoke testing	8/8 pruebas aprobadas	Aprobado
API Auth Sprint 1	8/8 pruebas aprobadas	Aprobado
Backend + API Integration	13/13 Jest y 20/20 Newman aprobados	Aprobado
Cypress E2E basico	6/6 pruebas aprobadas	Aprobado
Cypress Component Testing	Fallos por Router, Providers y props indefinidas	Condicional
Rendimiento/carga	Parcial; bloqueado por rate limiter	No concluyente
Seguridad estatica	Hallazgos de validacion, timing attack, tipado y sanitizacion	Requiere correccion
Accesibilidad	13/13 E2E de accesibilidad fallidas	No aprobado
Cobertura de codigo	Cobertura global baja y critica en hooks/services	Insuficiente
Requisitos y diseno	Omisiones y ambiguedades relevantes	Requiere cierre

Fuentes revisadas

#	Archivo	Tipo de evidencia
1	INFORME_prueba_carga.pdf	Carga, stress, spike y limitaciones por rate limiter
2	Informe_Pruebas_COberutra (1).docx	Cobertura frontend, pruebas unitarias y componentes
3	informe_pruebas_estaticas.pdf	Analisis estatico de codigo, requisitos y diseno
4	REPORTE_ACCESIBILIDAD_FINAL (1).docx	Accesibilidad WCAG 2.1 AA, axe, NVDA, Lighthouse
5	Informe_Pruebas_Integracion_VibePulse_v2.docx	Integracion backend, API, frontend component testing y E2E
6	informe de errores.docx	Cobertura de codigo; contiene inconsistencias con el dominio del proyecto
7	Informe_Rendimiento_VibePulse.docx	Rendimiento basico con k6 sobre 27 endpoints
8	informe_pruebas_QA_VibePulse (1).docx	Pruebas responsive/mobile; excluidas de este informe por indicacion del solicitante
9	Informe_Smoke_VibePulse.pdf	Smoke testing backend y build frontend
10	Informe_Pruebas_API_Ecommerce_Final.pdf	Pruebas API del modulo autenticacion/registro

Nota QA: el archivo `informe de errores.docx` contiene referencias a C++, firmware, sensores, bateria, Bluetooth y gestion energetica. Ese contenido no corresponde directamente al e-commerce VibePulse. Por tanto, se toma solo como referencia general de cobertura, no como evidencia principal del producto.

Criterios de consolidacion

1. Se eliminaron hallazgos duplicados entre documentos. Por ejemplo, el problema de rate limiter aparece en informes de carga, rendimiento e integracion; se registra como un unico defecto consolidado.
2. Se excluyeron hallazgos responsive/mobile por instruccion directa: las pruebas responsive ya fueron solucionadas.
3. Se priorizaron defectos con impacto real en calidad, seguridad, confiabilidad, accesibilidad, mantenibilidad e integracion.
4. Se agruparon los problemas en cinco bloques de trabajo para distribuirlos entre cinco ingenieros de software.
5. Cada bloque contiene: objetivo, alcance, defectos asignados, evidencia, pasos de reproduccion, archivos relacionados, entregables esperados y criterios de aceptacion.

Inventario de endpoints evaluados

Endpoints publicos

Metodo	Ruta	Estado observado
GET	/api/health	Aprobado
GET	/api/products	Aprobado en smoke/API/rendimiento parcial
GET	/api/products/featured	Aprobado
GET	/api/products/:id	Aprobado
GET	/api/categories	Aprobado
GET	/api/categories/:slug	Aprobado
POST	/api/auth/register	Funcional, pero afectado por rate limiter en carga
POST	/api/auth/login	Funcional, pero afectado por rate limiter en carga

Endpoints autenticados

Metodo	Ruta	Estado observado
GET	/api/auth/me	Aprobado en API; bloqueado en rendimiento por falta de token

Metodo	Ruta	Estado observado
GET	/api/auth/profile	Aprobado en API; bloqueado en rendimiento por falta de token
GET	/api/cart	Protegido correctamente; rendimiento no concluyente
POST	/api/cart/items	Integracion aprobada; rendimiento no concluyente
PATCH	/api/cart/items/:id	Integracion aprobada; rendimiento no concluyente
DELETE	/api/cart/items/:id	Integracion aprobada; rendimiento no concluyente
POST	/api/orders	Integracion aprobada; rendimiento no concluyente
GET	/api/orders	Integracion aprobada; rendimiento no concluyente

Endpoints administrativos

Metodo	Ruta	Estado observado
GET	/api/auth/admin-only	Control de rol aprobado
GET	/api/admin/dashboard /metrics	Aprobado, pero es el endpoint admin mas lento
GET	/api/admin/users	Aprobado
GET	/api/admin/orders	Aprobado
PATCH	/api/admin/orders/:id /status	Aprobado
POST	/api/products	Aprobado parcialmente
PUT	/api/products/:id	Bloqueado en algunas pruebas por dependencia de ID

Metodo	Ruta	Estado observado
DELETE	/api/products/:id	Bloqueado en algunas pruebas por dependencia de ID
POST	/api/categories	Aprobado
PUT	/api/categories/:id	Aprobado
DELETE	/api/categories/:id	Aprobado

Matriz consolidada de hallazgos

ID	Hallazgo consolidado	Severidad	Estado	Bloque responsable
QA-001	Rate limiter bloquea pruebas de carga/rendimiento y genera HTTP 429	Alta	Abierto	Bloque 1
QA-002	Falta de metricas reales en Cart y Orders por cascada de token nulo	Alta	Abierto	Bloque 1
QA-003	Endpoint admin dashboard metrics con P95 mayor al resto del modulo	Media	Abierto	Bloque 1
QA-004	Condicion de carrera en inventario/carrito	Critica	Abierto	Bloque 2
QA-005	Validaciones insuficientes y sanitizacion incompleta	Alta	Abierto	Bloque 2

ID	Hallazgo consolidado	Severidad	Estado	Bloque responsable
QA-006	Riesgo de timing attack y enumeracion de usuarios	Alta	Abierto	Bloque 2
QA-007	Tipado inseguro y coerciones debiles en TypeScript	Media	Abierto	Bloque 2
QA-008	Fallos de Cypress Component por ausencia de Router	Alta	Abierto	Bloque 3
QA-009	Fallos de Cypress Component por Providers/OutletContext faltantes	Alta	Abierto	Bloque 3
QA-010	Props indefinidas en componentes de datos	Alta	Abierto	Bloque 3
QA-011	Violacion de clave foranea al limpiar datos de pruebas	Critica	Abierto	Bloque 3
QA-012	Cobertura baja en hooks y services	Alta	Abierto	Bloque 4
QA-013	Mocks ausentes para dashboard/admin en	Alta	Abierto	Bloque 4

ID	Hallazgo consolidado pruebas frontend	Severidad	Estado	Bloque responsable
QA-014	Accesibilidad critica: button-name, select-name y contraste	Alta	Abierto	Bloque 4
QA-015	Landmarks, headings y foco visual insuficientes	Media	Abierto	Bloque 4
QA-016	Modelos Prisma incompletos en requisitos	Critica	Abierto	Bloque 5
QA-017	Estrategia de sesion/token no documentada	Alta	Abierto	Bloque 5
QA-018	Reglas de carrito, stock, orden y pago no definidas	Alta	Abierto	Bloque 5
QA-019	Falta de trazabilidad requisito-diseno-prueba	Alta	Abierto	Bloque 5
QA-020	Falta de diagramas, contratos de datos y criterios de error	Media	Abierto	Bloque 5

Bloque 1 - Rendimiento, carga, rate limiting y estabilidad operativa

Responsable sugerido

Ingeniero 1 - Backend Performance / DevOps QA

Objetivo

Garantizar que las pruebas de carga y rendimiento produzcan metricas reales, repetibles y tecnicamente validas. Este bloque debe resolver la interferencia del rate limiter en entornos de prueba y completar la medicion de los endpoints que quedaron bloqueados.

Hallazgos asignados

ID	Hallazgo	Severidad
QA-001	Rate limiter bloquea pruebas y genera HTTP 429	Alta
QA-002	Cart y Orders sin metricas reales por token nulo	Alta
QA-003	admin/dashboard/metrics es el endpoint admin mas lento	Media

Evidencia consolidada

Los informes de carga y rendimiento reportan que el middleware de rate limiting responde con **HTTP 429 Too Many Requests** cuando varias solicitudes salen desde la misma IP local. Esto afecta especialmente los flujos de autenticacion (`register`, `login`) y causa una cascada: al no obtenerse `clientToken`, no se ejecutan correctamente los modulos Cart, Orders ni endpoints Auth autenticados.

Datos relevantes reportados:

Metrica	Valor observado
Errores HTTP 429	6 requests
Tasa global de error	25.58%
Endpoints bloqueados	13 de 27
Endpoints ejecutados con trafico real	14 de 27
Errores HTTP 500	0
Thresholds de tiempo	36/36 aprobados
Cart operations	0 por token nulo
Orders operations	0 por token nulo
P95 admin/dashboard/metrics	167 ms

Pasos para reproducir QA-001 y QA-002

1. Levantar backend en `http://localhost:4000`.
2. Ejecutar seed de base de datos.

3. Ejecutar script k6 con escenario de 2 o mas usuarios virtuales.
4. El script intenta flujo de autenticacion:
   - `POST /api/auth/register`
   - `POST /api/auth/login`
5. Tras varias peticiones desde la misma IP, el servidor responde `429 Too Many Requests`.
6. El `clientToken` queda nulo.
7. Los grupos posteriores de Cart y Orders no se ejecutan realmente.

Archivos y componentes relacionados

Elemento	Ubicacion sugerida
Middleware rate limiter	`rateLimiter.ts`
Script de rendimiento	`test-rendimiento.js`
Configuracion de entorno	`.env`, `NODE_ENV`
Seed de datos	`server/prisma/seed.ts`
Modulos afectados	Auth, Cart, Orders, Products, Admin

Correcciones requeridas

1. Implementar configuracion diferenciada por ambiente.
2. Agregar whitelist para IPs locales en pruebas: `127.0.0.1` y `::1`.
3. Ajustar limites solo cuando `NODE_ENV === 'test'`.
4. Mantener configuracion restrictiva en produccion.
5. Re-ejecutar k6 y documentar resultados completos de los 27 endpoints.
6. Agregar fallback de `productId` desde seed para evitar bloqueo de PUT/DELETE Products.
7. Evaluar cache de 30 segundos para `/api/admin/dashboard/metrics`.

Ejemplo de ajuste recomendado

```
const isTest = process.env.NODE_ENV === 'test';

export const apiLimiter = rateLimit({
  windowMs: isTest ? 60_000 : 15 * 60_000,
  max: isTest ? 500 : 100,
  skip: (req) => {
    if (!isTest) return false;
    return ['127.0.0.1', '::1', '::ffff:127.0.0.1'].includes(req.ip || '');
  },
});
```

Entregables del Bloque 1

Entregable	Criterio de aceptacion
Rate limiter ajustado por ambiente	No hay HTTP 429 en pruebas locales controladas

Entregable	Criterio de aceptacion
Re-ejecucion k6 completa	27/27 endpoints con metrica real o justificacion valida
Reporte actualizado de rendimiento	Incluye P95, P99, tasa de error, VUs, iteraciones y endpoints bloqueados = 0
Dashboard metrics optimizado o justificado	P95 se mantiene bajo umbral y con margen razonable
Configuracion documentada	README explica comportamiento de rate limiter en test y produccion

Bloque 2 - Seguridad backend, validaciones, sanitizacion e integridad transaccional

Responsable sugerido

Ingeniero 2 - Backend Security / Data Integrity

Objetivo

Corregir riesgos de seguridad, validacion, sanitizacion e integridad de datos encontrados en el analisis estatico de codigo. Este bloque atiende defectos que pueden causar vulnerabilidades, datos invalidos o ventas inconsistentes.

Hallazgos asignados

ID	Hallazgo	Severidad
QA-004	Condicion de carrera en inventario/carrito	Critica
QA-005	Validaciones insuficientes y sanitizacion incompleta	Alta
QA-006	Riesgo de timing attack y enumeracion de usuarios	Alta
QA-007	Tipado inseguro y coerciones debiles en TypeScript	Media

Defecto QA-004 - Condicion de carrera en inventario/carrito

Descripcion

En `routes/cart.ts`, la verificacion de stock y la actualizacion posterior del inventario se ejecutan en operaciones separadas. Si dos solicitudes concurrentes intentan comprar o reservar el mismo producto, ambas pueden leer stock disponible antes de que una actualice la cantidad. Esto puede producir **overselling** o inventario negativo.

Pasos para reproducir

1. Crear o identificar un producto con stock bajo, por ejemplo 1 unidad.
2. Abrir dos sesiones o ejecutar dos requests concurrentes hacia el endpoint de carrito/compra.
3. Enviar simultaneamente dos solicitudes para agregar o comprar el producto.
4. Ambas solicitudes leen stock disponible.
5. Ambas continuan el flujo.
6. El sistema puede permitir mas ventas que unidades disponibles.

Archivos relacionados

Archivo	Area
`routes/cart.ts`	Gestion de carrito e inventario
`schema.prisma`	Modelo de producto, variante, carrito
`cart.integration.test.ts`	Pruebas de carrito

Correccion recomendada

Usar una transaccion atomica para validar y descontar inventario.

```
await prisma.$transaction(async (tx) => {
  const product = await tx.product.findUnique({ where: { id: productId } });

  if (!product || product.stock < quantity) {
    throw new Error('Stock insuficiente');
  }

  await tx.product.update({
    where: { id: productId },
    data: { stock: { decrement: quantity } },
  });

  await tx.cartItem.create({
    data: {
      cartId,
      productId,
      quantity,
    },
```

```
  });
});
```

Defecto QA-005 - Validaciones y sanitizacion insuficientes

Descripcion

Se identificaron validaciones debiles en formularios y controladores. Los campos `name`, `email`, `color`, `size`, `productId`, `variantId`, `price`, `comparePrice`, `stock` y `categoryId` requieren validacion y sanitizacion mas estricta.

Ejemplos observados

Archivo	Problema
`authControllerRegister.ts`	`name` y `email` aceptan cadenas sin sanitizacion suficiente
`authControllerRegister.ts`	Password solo valida longitud minima
`productController.ts`	Uso de `Number()` sin validar `NaN`, `Infinity`, negativos o decimales invalidos
`routes/cart.ts`	Inputs de carrito sin validacion robusta
`middlewares/validation.ts`	Regex de email demasiado basico

Correccion recomendada

1. Implementar esquemas de validacion con Zod, Joi, Yup o express-validator.
2. Centralizar validaciones por DTO.
3. Normalizar email con `trim().toLowerCase()`.
4. Sanitizar texto antes de almacenarlo o renderizarlo.
5. Validar numeros con `Number.isFinite` y reglas de negocio.

Ejemplo:

```
const parsedPrice = Number(price);

if (!Number.isFinite(parsedPrice) || parsedPrice < 0) {
  return res.status(400).json({
    success: false,
    message: 'Precio invalido',
  });
}
```

Defecto QA-006 - Timing attack y enumeracion de usuarios

Descripcion

El login responde mas rapido cuando el usuario no existe que cuando el usuario existe pero la contrasena es incorrecta, porque en el segundo caso ejecuta bcrypt. Esto puede permitir inferir si un correo existe midiendo tiempos de respuesta. Tambien se reporta que el registro informa explicitamente cuando un correo ya esta registrado.

Archivos relacionados

Archivo	Problema
`authControllerLogin.ts`	Diferencias temporales entre usuario inexistente y password incorrecto
`authControllerRegister.ts`	Mensaje explicito de correo ya registrado

Correccion recomendada

1. Usar mensaje generico en login y registro.
2. Ejecutar comparacion bcrypt con hash dummy cuando el usuario no exista.
3. Mantener logs internos para depuracion, sin exponer informacion al usuario.

Ejemplo de mensaje generico:

```
{
  "success": false,
  "message": "No fue posible completar la operacion con los datos
proporcionados."
}
```

Defecto QA-007 - Tipado inseguro y coerciones debiles

Descripcion

Se encontro uso de `catch(error: any)` y coerciones de tipo como `as 'CLIENT'`. Esto reduce el valor de TypeScript y puede ocultar errores.

Correccion recomendada

```
catch (error: unknown) {
  next(error);
}
```

Ademas, reemplazar coerciones por validaciones explicitas de enums o tipos.

Entregables del Bloque 2

Entregable	Criterio de aceptacion
Transaccion atomica de inventario	No se produce overselling en solicitudes concurrentes
Validaciones centralizadas	Todos los inputs criticos tienen esquema documentado y testeado
Sanitizacion de entradas	Campos textuales se normalizan y sanitizan
Mitigacion timing attack	Login no permite inferir existencia de usuario por tiempos o mensajes
Tipado seguro	Eliminacion de `any` innecesario en catch y controladores
Pruebas negativas agregadas	Casos para `NaN`, negativos, strings maliciosos, password debil y email invalido

Bloque 3 - Automatizacion de pruebas, integracion frontend y estabilidad de suites

Responsable sugerido

Ingeniero 3 - QA Automation / Frontend Integration Testing

Objetivo

Corregir los fallos de pruebas automatizadas que impiden validar componentes frontend de forma aislada y mejorar la confiabilidad de las suites Cypress/Jest.

Hallazgos asignados

ID	Hallazgo	Severidad
QA-008	Ausencia de Router en Cypress Component	Alta
QA-009	Providers y OutletContext faltantes	Alta
QA-010	Props indefinidas en componentes de datos	Alta
QA-011	Violacion de clave foranea al limpiar datos de pruebas	Critica

Estado actual de integracion

Capa	Resultado
Jest + Supertest backend	13/13 PASS
Newman API	20/20 PASS
Cypress E2E	6/6 PASS
Cypress Component Testing	Fallos por configuracion de contexto

Defecto QA-008 - Componentes sin Router

Descripcion

Varios componentes usan hooks de React Router como `useNavigate`, `useLocation` o `useRoutes`. En Cypress Component Testing se montan de forma aislada sin `<Router>`, generando errores.

Componentes afectados

- `ProductCard`
- `MainStoreLayout`
- `AdminLayout`
- `AdminDashboardPage`
- `Admin`
- `Cart`
- `Catalog`
- `Checkout`
- `ConfirmationPage`
- `Home`
- `Login`
- `ProcessingPaymentPage`
- `ProductDetail`
- `PurchaseAlertPage`
- `Register`
- `App`

Pasos para reproducir

1. Ir a `/client`.
2. Ejecutar `npx cypress run --component`.
3. Montar un componente que use `useNavigate()` sin wrapper.
4. Se obtiene error: `useNavigate() may be used only in the context of a <Router> component`.

Correccion recomendada

Crear un helper reutilizable de montaje:

```
import { MemoryRouter } from 'react-router-dom';

export function mountWithRouter(ui: React.ReactNode, initialPath = '/') {
  return cy.mount(
    <MemoryRouter initialEntries={[initialPath]}>
      {ui}
    </MemoryRouter>
  );
}
```

Defecto QA-009 - Providers y OutletContext faltantes

Descripcion

Algunos componentes dependen de contextos globales o contextos entregados por layouts padres. Al montarlos sin el provider correspondiente, fallan.

Componentes afectados

Componente	Dependencia faltante
`OrderForm`	`CheckoutProvider`
`AdminOrdersPage`	`useOutletContext` desde `AdminLayout`
`AdminProductsPage`	`useOutletContext` desde `AdminLayout`
`AdminUsersPage`	`useOutletContext` desde `AdminLayout`

Correccion recomendada

Crear helpers por dominio:

```
cy.mount(
  <MemoryRouter>
    <CheckoutProvider>
      <OrderForm />
    </CheckoutProvider>
  </MemoryRouter>
);
```

Para admin, usar mock controlado de `useOutletContext` o montar con `AdminLayout`.

Defecto QA-010 - Props indefinidas

Componentes afectados

Componente	Error observado
`CatalogFilters`	Acceso a `categoryId` de undefined
`ProductGrid`	Acceso a `length` de undefined

Componente	Error observado
`DashboardCard`	Uso de `map` sobre undefined

Correccion recomendada

1. Crear fixtures en `client/cypress/fixtures/`.
2. Pasar props explicitas en cada test.
3. Agregar defaults seguros en componentes.

Ejemplo:

```
const mockCategories = [
  { id: 'c1', categoryId: 'cat-1', name: 'Remeras' },
  { id: 'c2', categoryId: 'cat-2', name: 'Pantalones' },
];

cy.mount(<CatalogFilters categories={mockCategories} />);
```

Defecto QA-011 - Violacion de clave foranea en limpieza de datos

Descripcion

`server/__tests__/auth.test.ts` intenta ejecutar `prisma.user.deleteMany()` sin eliminar primero registros dependientes en `Cart` y `CartItem`. PostgreSQL bloquea la operacion por la restriccion `Cart_userId_fkey`.

Pasos para reproducir

1. Ir a `/server`.
2. Ejecutar `npm run test`.
3. Observar fallo en `auth.test.ts`.
4. El error indica violacion de clave foranea por dependencias de carrito.

Correccion recomendada

```
await prisma.cartItem.deleteMany();
await prisma.cart.deleteMany();
await prisma.user.deleteMany();
```

Entregables del Bloque 3

Entregable	Criterio de aceptacion
Helper `mountWithRouter`	Todos los componentes con React Router montan sin error
Helpers de Providers	Checkout y Admin subpages montan con contexto valido
Fixtures frontend	`CatalogFilters`, `ProductGrid`, `DashboardCard` no fallan por undefined

Entregable	Criterio de aceptacion
Limpieza DB corregida	`auth.test.ts` pasa sin violar claves foraneas
Suite Cypress Component estable	100% de componentes definidos ejecutan PASS o fallan por defecto real, no por configuracion
Documentacion de testing	README con patrones para Router, Providers, fixtures y seed

Bloque 4 - Cobertura, accesibilidad y calidad frontend no funcional

Responsable sugerido

Ingeniero 4 - Frontend Quality / Accessibility / Coverage

Objetivo

Elevar la calidad frontend mediante mejoras de cobertura, mocks, accesibilidad y pruebas negativas. Este bloque no incluye defectos responsive/mobile porque ya fueron solucionados y quedaron fuera del alcance por solicitud.

Hallazgos asignados

ID	Hallazgo	Severidad
QA-012	Cobertura baja en hooks y services	Alta
QA-013	Mocks ausentes para dashboard/admin	Alta
QA-014	Accesibilidad critica: button-name, select-name, color-contrast	Alta
QA-015	Landmarks, headings y foco visual insuficientes	Media

Cobertura de codigo reportada

Metrica	Resultado
Statements	33.69%
Branches	35.39%
Functions	30.00%
Lines	35.11%

Cobertura por modulo

Modulo	Cobertura
Components	75.00%
Pages	61.70%
Context	43.47%
Hooks	7.29%
Services	7.14%
Utils	92.85%

Defecto QA-012 - Cobertura baja en hooks y services

Descripcion

Aunque varias pruebas pasan, la cobertura real de hooks y servicios es muy baja. Esto deja sin validar ramas criticas: errores de red, respuestas 401/403, token expirado, estados vacios, errores 500, orden inexistente, carrito vacio y validaciones negativas.

Correcciones recomendadas

1. Agregar pruebas unitarias para hooks.
2. Agregar pruebas de servicios API con mocks de fetch/axios.
3. Cubrir casos negativos y de error.
4. Definir umbral minimo progresivo:
   - fase 1: 50%
   - fase 2: 65%
   - fase 3: 80%

Defecto QA-013 - Mocks ausentes para dashboard/admin

Descripcion

Las pruebas del dashboard fallaban por depender de llamadas reales a:

- `getAdminDashboardMetricsRequest`
- `getAdminOrdersRequest`

Correccion recomendada

```
vi.mock('@/services/admin', () => ({
  getAdminDashboardMetricsRequest: vi.fn().mockResolvedValue(mockMetrics),
  getAdminOrdersRequest: vi.fn().mockResolvedValue(mockOrders),
}));
```

Defecto QA-014 - Accesibilidad critica

Estado reportado

Suite	Resultado
Componentes accesibilidad	5/5 PASS
E2E accesibilidad con axe	13/13 FAIL

Problemas principales

Problema	Impacto	Severidad
`button-name`	Lector de pantalla no identifica botones	Alta
`select-name`	Selectores sin nombre accesible	Alta
`color-contrast`	Baja legibilidad para usuarios con baja vision	Alta
Botones sociales no tabulables	Login inaccesible por teclado	Alta

Rutas evaluadas

- `/login`
- `/register`
- `/home`
- `/catalogo`
- `/products/:id`
- `/cart`
- `/checkout`
- `/admin`
- `/admin/products`
- `/admin/orders`
- `/admin/users`
- `/admin/settings`

Correcciones recomendadas

1. Agregar texto visible o `aria-label` a botones iconograficos.
2. Asociar `label` con `select` e inputs.
3. Corregir contraste de color segun WCAG 2.1 AA.
4. Hacer tabulables botones sociales de login.
5. Validar con axe y Lighthouse despues de cada correccion.

Defecto QA-015 - Landmarks, headings y foco visual

Descripcion

Se reportan problemas de estructura semantica: ausencia o mala jerarquia de encabezados, regiones sin landmark adecuado, falta de un `h1` por pagina y foco visual poco perceptible.

Correcciones recomendadas

1. Garantizar un solo `<main>` por pantalla.
2. Agregar un `<h1>` descriptivo por vista principal.
3. Mantener jerarquia `h1 > h2 > h3` sin saltos arbitrarios.
4. Mejorar foco con estilos visibles:

```
className="focus:outline-none focus:ring-2 focus:ring-offset-2"
```

Entregables del Bloque 4

Entregable	Criterio de aceptacion
Plan de cobertura	Hooks y services aumentan cobertura al menos a 50% inicial
Mocks admin	Dashboard/admin testea sin depender de API real
Correccion accesibilidad critica	axe no reporta `button-name`, `select-name` ni contrastes graves
Navegacion por teclado	Tab y Shift+Tab recorren controles principales
Validacion NVDA/Lighthouse	Reporte posterior confirma mejora
Umbrales CI	Pipeline falla si cobertura baja del umbral acordado

Bloque 5 - Requisitos, diseno, trazabilidad y documentacion tecnica

Responsable sugerido

Ingeniero 5 - Requirements / Architecture / QA Governance

Objetivo

Cerrar vacios de requisitos y diseno que estan causando ambiguedades tecnicas e integracion inconsistente. Este bloque debe producir documentacion clara y trazable para que desarrollo y QA trabajen sobre los mismos contratos.

Hallazgos asignados

ID	Hallazgo	Severidad
QA-016	Modelos Prisma incompletos	Critica
QA-017	Estrategia de sesion/token no documentada	Alta
QA-018	Reglas de carrito, stock, orden y pago no definidas	Alta
QA-019	Falta de trazabilidad requisito-diseno-prueb a	Alta
QA-020	Falta de diagramas, contratos de datos y criterios de error	Media

Defecto QA-016 - Modelos Prisma incompletos

Descripcion

La documentacion de requisitos menciona entidades como `Product`, `Cart`, `CartItem` y `Order`, pero no define todos sus campos, tipos, relaciones ni restricciones.

Impacto

Cada equipo puede implementar estructuras diferentes, generando errores de integracion, consultas incompatibles y deuda tecnica.

Entregable requerido

Crear una seccion formal de modelo de datos con:

- `User`
- `Product`
- `ProductVariant`, si aplica
- `Category`
- `Cart`
- `CartItem`
- `Order`
- `OrderItem`
- enums de rol y estado
- relaciones y restricciones

Defecto QA-017 - Estrategia de sesion/token no documentada

Descripcion

Los sprints asumen usuario autenticado, pero no documentan donde se guarda el token, como se envia, como se renueva, cuando expira y que ocurre si la sesion caduca.

Definiciones requeridas

Tema	Decision requerida
Tipo de token	JWT
Expiracion	7 dias reportados en API; confirmar si se mantiene
Almacenamiento frontend	localStorage, sessionStorage o cookie httpOnly
Header	`Authorization: Bearer <token>`
Token expirado	Redireccion a login y mensaje de sesion expirada
Rutas protegidas	Lista de rutas cliente/admin
Roles	CLIENT y ADMIN

Defecto QA-018 - Reglas de carrito, stock, orden y pago no definidas

Problemas detectados

Area	Regla faltante
Carrito	Que pasa si se agrega dos veces el mismo producto
Carrito	Limite por stock disponible
Carrito	Persistencia al cerrar navegador o expirar sesion
Checkout	Formula subtotal, impuestos, envio y total
Pago	Metodos simulados disponibles
Ordenes	Estados posibles y transiciones
Admin	CRUD completo de productos

Propuesta de estados de pedido

Estado	Descripcion	Transiciones sugeridas
PENDING	Orden creada, pendiente de procesamiento	PROCESSING, CANCELLED
PROCESSING	Orden en preparacion	SHIPPED, CANCELLED
SHIPPED	Orden enviada	DELIVERED

Estado	Descripcion	Transiciones sugeridas
DELIVERED	Orden entregada	Ninguna
CANCELLED	Orden cancelada	Ninguna

Defecto QA-019 - Falta de trazabilidad

Descripcion

No existe matriz formal que relacione requisitos, pantallas, componentes, endpoints y casos de prueba.

Matriz minima recomendada

Requisito	Pantalla/componente	Endpoint	Caso de prueba	Estado
RF-AUTH-001 Login cliente	`Login.tsx`	`POST /api/auth/login`	API-03, E2E-001	Aprobado
RF-CART-001 Agregar al carrito	`Cart.tsx`	`POST /api/cart/items`	INT-CART-001	Pendiente rendimiento
RF-ORDER-001 Crear orden	`Checkout.tsx`	`POST /api/orders`	INT-ORDER-001	Pendiente rendimiento
RF-ADMIN-001 Ver metricas	`AdminDashboardPage.tsx`	`GET /api/admin/dashboard/metrics`	INT-ADMIN-001	Aprobado con observacion

Defecto QA-020 - Falta de diagramas, contratos y criterios de error

Artefactos requeridos

1. Diagrama de casos de uso.
2. Diagrama de secuencia del flujo de compra.
3. Diagrama de componentes frontend/backend.
4. Contratos JSON de endpoints principales.
5. Politica de errores API.
6. Guia de variables de entorno.
7. Matriz de permisos por rol.
8. Criterios de aceptacion para errores de red, 401, 403, 404 y 500.

Entregables del Bloque 5

Entregable	Criterio de aceptacion
Modelo de datos formal	Todos los modelos usados por sprints estan definidos

Entregable	Criterio de aceptacion
Documento de sesion/token	Desarrollo y QA conocen almacenamiento, expiracion y rutas protegidas
Reglas de carrito/orden/pago	Casos de borde definidos y testeables
Matriz de trazabilidad	Cada requisito tiene pantalla, endpoint y caso de prueba
Diagramas minimos	Casos de uso, secuencia de compra y componentes
Politica de errores API	Formato JSON comun para errores y mensajes

Plan de ejecucion recomendado para el equipo

Semana	Actividad	Responsable principal	Resultado esperado
1	Ajustar rate limiter y re-ejecutar k6	Ingeniero 1	Rendimiento completo de 27 endpoints
1	Corregir transacciones de inventario y validaciones criticas	Ingeniero 2	Seguridad e integridad backend mejoradas
1	Corregir Cypress Component Testing	Ingeniero 3	Suite frontend estable
2	Incrementar cobertura y corregir accesibilidad critica	Ingeniero 4	Reporte axe/Lighthouse mejorado
2	Cerrar requisitos, trazabilidad y contratos	Ingeniero 5	Documentacion de producto consistente
3	Regresion completa	Todos	Evidencia consolidada de correccion
3	Informe final de cierre QA	Lider QA	Veredicto final actualizado

Criterios de salida QA

El sistema puede considerarse listo para una entrega formal si cumple los siguientes criterios:

Criterio	Umbral minimo
Smoke suite	100% PASS
API Auth	100% PASS
Integracion backend/API	100% PASS
Cypress Component	Sin fallos por configuracion
E2E basico	100% PASS
Rendimiento	27/27 endpoints con medicion real o exclusion justificada
Rate limiter	Sin interferencia en entorno de pruebas
Errores HTTP 500	0 en suites controladas
Accesibilidad critica	Sin fallos `button-name`, `select-name`, contraste grave
Cobertura hooks/services	Minimo 50% inicial; plan a 80%
Inventario/carrito	Sin overselling en concurrencia
Requisitos	Modelos, estados, tokens y reglas de negocio definidos
Trazabilidad	Requisito -> componente -> endpoint -> prueba

Riesgos residuales

Riesgo	Probabilidad	Impacto	Nivel	Mitigacion
Rate limiter sigue bloqueando pruebas	3	5	Alto	Configuracion por ambiente y whitelist test
Inventario inconsistente por concurrencia	3	5	Alto	Transacciones atomicas y pruebas concurrentes

Riesgo	Probabilidad	Impacto	Nivel	Mitigacion
Accesibilidad no corregida	4	4	Alto	Axe en CI y validacion manual NVDA
Cobertura baja oculta errores	4	4	Alto	Umbrales progresivos y pruebas negativas
Requisitos ambiguos generan retrabajo	4	5	Muy alto	Matriz de trazabilidad y contratos de datos
Mocks ausentes hacen tests fragiles	3	3	Medio	Fixtures y mocks estandarizados

Recomendacion final QA

El sistema **VibePulse** debe avanzar con una estrategia de correccion por bloques. No se recomienda tratar los hallazgos como errores aislados, porque varios tienen relacion causal: los requisitos ambiguos generan implementaciones inconsistentes; las suites mal configuradas impiden validar frontend; el rate limiter invalida metricas de rendimiento; y la baja cobertura oculta errores en escenarios negativos.

La recomendacion es ejecutar primero los bloques 1, 2 y 3, porque habilitan mediciones confiables y estabilidad tecnica. En paralelo, el bloque 5 debe cerrar contratos y reglas de negocio. Luego, el bloque 4 debe elevar cobertura y accesibilidad para completar una entrega con calidad defendible ante evaluacion academica o tecnica.

Veredicto profesional: funcionalidad base aprobada, calidad final condicionada a la correccion de hallazgos criticos y altos.

Referencias metodologicas

1. IBM Documentation. “Test reports for the Engineering Test Management integration.” IBM Engineering Lifecycle Management / DOORS. Consultado para estructurar el reporte alrededor de cobertura, estado, veredicto y registros de error.

2. IBM Documentation. “Engineering Test Management predefined reports.” Consultado para la orientacion de reportes por categorias: defectos, requisitos, casos de prueba y ejecuciones.
3. ISTQB Glossary. “Test Summary Report.” Consultado como referencia conceptual para informes que resumen actividades, resultados y evaluacion contra criterios de salida.
4. ISTQB Glossary. “Test Completion Report.” Consultado como referencia para reportes al cierre de hitos de prueba.

Anexo A - Pruebas base aprobadas

Smoke testing aprobado

Caso	Resultado
Variables `.env`: `DATABASE_URL`, `JWT_SECRET`, `PORT`	PASS
`GET /api/health`	PASS
Conectividad DB con `GET /api/products`	PASS
Ruta protegida `/api/cart` sin token devuelve 401	PASS
Login vacio devuelve 400	PASS
`/api/admin` registrado y protegido	PASS
`/api/orders` registrado y protegido	PASS
Build frontend con TypeScript + Vite	PASS

API Auth Sprint 1 aprobado

Caso	Endpoint	Resultado
Verificacion servidor	`GET /api/health`	PASS
Registro valido	`POST /api/auth/register`	PASS
Login valido	`POST /api/auth/login`	PASS
Usuario con token	`GET /api/auth/me`	PASS
Usuario sin token	`GET /api/auth/me`	PASS 401
CLIENT en admin-only	`GET /api/auth/admin-only`	PASS 403
Login password incorrecto	`POST /api/auth/login`	PASS 401
Perfil autenticado	`GET /api/auth/profile`	PASS

Anexo B - Correcciones ya resueltas en smoke

Defecto	Causa	Correccion aplicada
Prisma P1000	`localhost` resolvia a IPv6 `::1` en Windows	Usar `127.0.0.1` en `DATABASE_URL`
`JWT_SECRET undefined`	`dotenv.config()` se ejecutaba tarde	Mover `dotenv.config()` al inicio del entrypoint
Assertion incorrecto en products	Test esperaba array plano	Ajustar a `{ success: true, data: [] }`

Anexo C - Exclusiones conscientes

Exclusion	Motivo
Defectos responsive/mobile reportados en `informe_pruebas_QA_VibePulse (1).docx`	El solicitante informo que ya fueron solucionados y pidio no incluirlos
Evidencia tecnica de firmware/sensores en `informe de errores.docx`	No corresponde al dominio funcional de VibePulse e-commerce
Pruebas de pasarela de pago externa real	No habia cuentas sandbox externas disponibles
Pen testing profundo OWASP ZAP	Fuera del alcance de los informes revisados